Reproducing Native Sparse Attention

Hardware-Aligned Sparse Attention for Long-Context LLMs

Yentl Collin / Gaspard Beaudoin

MVA — Large Scale Models

ENS Paris-Saclay, 2025–2026

github.com/YentlCollin/native-sparse-attention

Abstract. Long-context language models are bottlenecked by the $\mathcal{O}(n^2)$ cost of dense attention. *Native Sparse Attention* (NSA, DeepSeek-AI, arXiv:2502.11089) proposes a three-branch sparse architecture — **compression**, **selection**, and **sliding window** — whose block-structured memory access is natively aligned with modern GPU tensor cores, enabling end-to-end differentiable training without post-hoc pruning. We reproduce two core experimental claims on a real A100-SXM4-40GB: (1) the forward pass reaches **8.3**× speedup over dense attention at $n = 64$k tokens, exceeding the 5.8× theoretical FLOPs prediction thanks to cache-line-aligned block access; and (2) training three small transformers from scratch on a synthetic French NIAH task reveals that NSA reaches **97.6%** evaluation accuracy after 200 epochs, dramatically outperforming both Dense attention (**52.9%**) and a sliding window (**5.9%**) — demonstrating that NSA's inductive bias, not merely its efficiency, confers a learning advantage.

Introduction

The transformer's self-attention mechanism scales as $\mathcal{O}(n^2)$ in both computation and memory, making it the dominant bottleneck for long-context processing. At $n = 64{,}000$ tokens, a single BF16 attention layer requires ∼70 T-FLOP and consumes ∼6 GB of activations — prohibitive for either training or inference at scale. This has motivated an extensive line of work on *sparse attention*.

The hardware-alignment problem. Most sparse attention methods solve the *algorithmic* problem of reducing the number of attended positions, but fail to achieve *hardware* speedups because their irregular access patterns (scatter/gather across non-contiguous token indices) cannot exploit the coalesced DRAM reads that tensor-core pipelines require. Longformer [2], Reformer [5], and BigBird [3] achieve $\mathcal{O}(n)$ or $\mathcal{O}(n \log n)$ complexity on paper, yet their CUDA implementations are slower than FlashAttention in practice due to non-contiguous memory access or multi-pass computation. SparseTransformer [4] uses fixed strided patterns with no dynamic selection. More generally, token-granular selection methods require loading scattered individual tokens from the KV cache, preventing efficient adaptation of FlashAttention's blockwise tiling and forcing low hardware utilization.

NSA's design philosophy. *Native Sparse Attention* (NSA, DeepSeek-AI, Feb. 2025) addresses the hardware-alignment problem directly by constraining all sparse attention operations to act on *contiguous fixed-size blocks* of 64 tokens. This single constraint — which every branch of NSA shares — ensures that:

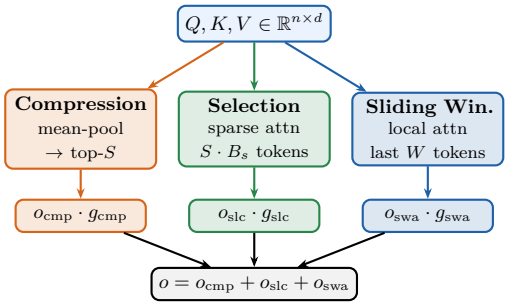
- GPU memory reads are always 64-token-aligned, maximizing L2 cache line reuse.
- The sparse pattern fits into standard FlashAttention tile loops without custom scatter logic.
- Gradients flow through all branches during training via standard backpropagation.

NSA is deployed in production in DeepSeek-R2 training and outperforms dense attention on long-context benchmarks (RULER, NIAH) while using a fraction of the compute.

This report. We reproduce two experimental claims: (1) arithmetic intensity and GPU speedup on an NVIDIA A100-SXM4-40GB (PyTorch 2.10, CUDA 12.8, BF16); and (2) end-to-end Needle-in-a-Haystack training comparing Dense, NSA, and Sliding Window, training three small transformers on a controlled synthetic benchmark on an L4 GPU (24 GB). We additionally train small LMs on WikiText-2 to verify language modeling quality.

NSA Architecture

Three-Branch Design



Learned gates $(g_{\text{cmp}}, g_{\text{slc}}, g_{\text{swa}})$ — end-to-end differentiable

The three branches partition the context into complementary regimes. The final output is a gated sum, enabling each branch to specialize during training.

Branch Details

Compression branch. Keys and values are mean-pooled into $C = \lceil n/B_s \rceil$ block representatives, each summarizing $B_s = 64$ tokens. A full attention pass over these $C \ll n$

compressed tokens produces: (a) a coarse output o_{cmp} capturing global block-level context, and (b) block relevance scores used to select the top-$S = 16$ blocks for the selection branch. This is the *only* component that processes the entire context, at dramatically reduced resolution.

Selection branch. Full-precision attention over $S \cdot B_s =$ 1,024 tokens from selected blocks. Crucially, each selected block is a *contiguous 64-token tile* in GPU memory — enabling the same coalesced DRAM-to-L2 transfer pattern as standard FlashAttention. No scatter/gather logic is required.

Sliding window branch. Standard causal local attention over the last W tokens via FlashAttention. Handles high-frequency recency patterns that coarse block selection may miss.

Complexity Analysis

Table 1 compares asymptotic costs. NSA's attention complexity is $\mathcal{O}(n \cdot n_{\text{active}})$ where $n_{\text{active}} = S \cdot B_s + W$ is *constant* in n, giving effectively linear scaling.

Table 1: **Complexity comparison.** n: sequence length, d: head dim, S: selected blocks, B_s: block size, W: window size, H: num heads.

Method	FLOPs	Memory	GPU-friendly?
Dense attention	$\mathcal{O}(n^2 dH)$	$\mathcal{O}(n^2 H)$	Yes (FA-2)
Longformer	$\mathcal{O}(ndH)$	$\mathcal{O}(nH)$	No (scatter)
Reformer	$\mathcal{O}(n \log n \cdot dH)$	$\mathcal{O}(nH)$	No (LSH)
NSA	$\mathcal{O}(n \cdot n_{\text{active}} \cdot dH)$	$\mathcal{O}(nH)$	**Yes (tiles)**

Arithmetic Intensity

Arithmetic intensity (AI) is the ratio between computation and memory traffic:

$$\text{AI} = \frac{\text{FLOPs}}{\text{bytes moved}}.$$

In practice, AI indicates whether a kernel is mainly memory-bound or compute-bound.

Let n be the sequence length and d the head dimension. For standard dense attention, the course slides give

$$\text{AI}_{\text{Attn}} = \frac{n(4d+5)}{4(d+n)},$$

hence, for long sequences ($n \gg d$), $\text{AI}_{\text{Attn}} \approx d$.

FlashAttention computes the same attention but avoids materializing the full $n \times n$ score matrix in HBM, greatly reducing memory traffic. Using the expression from the course slides, in the long-sequence regime:

$$\text{AI}_{\text{FlashAttn}} \approx 7n.$$

For NSA, each query attends only to an active set of size $n_{\text{active}} = SB_s + W$, so its arithmetic intensity depends on the sparse active set rather than the full sequence:

$$\text{AI}_{\text{NSA}} = O(n_{\text{active}}).$$

Overall, for large n:

$$\text{AI}_{\text{Attn}} \approx d, \qquad \text{AI}_{\text{FlashAttn}} \approx 7n, \qquad \text{AI}_{\text{NSA}} = O(n_{\text{active}}).$$

FlashAttention improves efficiency mainly by reducing I/O, while NSA also reduces the amount of computation by replacing attention over n tokens with attention over only n_{active} active tokens.

Arithmetic Intensity & GPU Speedup

NSA Kernel vs. FlashAttention

NSA's compression and sliding window branches are directly compatible with standard FlashAttention-2 kernels — their access patterns are already contiguous. The key challenge lies in the **selection kernel**.

Why FlashAttention's tiling strategy fails for sparse selection. FlashAttention-2 achieves high throughput by loading *temporally contiguous* query blocks (e.g. 64 consecutive queries) into on-chip SRAM, then streaming all KV blocks sequentially. For sparse selection, queries within such a block may require entirely *disjoint* KV blocks — making the outer-query-block loop inefficient: each query would trigger different, scattered memory fetches.

NSA's solution: group-centric loading. The NSA paper (§3.4) proposes a fundamentally different tiling strategy. Since modern LLMs use GQA, all query heads within a group share the *same* set of selected KV block indices $\mathcal{I}_t$. The kernel exploits this by loading the **entire GQA group at a single position** t into SRAM (rather than a block of consecutive positions sharing one head). The inner loop then fetches only the n selected contiguous KV blocks in sequence, eliminating redundant transfers. Formally, three properties characterize the kernel:

1. **Group-centric loading:** load all heads' queries $Q \in \mathbb{R}^{h \times d_k}$ and shared indices $\mathcal{I}_t$ for position t.
2. **Shared KV fetching:** iterate over contiguous blocks $K \in \mathbb{R}^{B_k \times d_k}$ indexed by $\mathcal{I}_t$, avoiding scattered reads.
3. **Outer loop on Triton grid:** since inner-loop length (proportional to selected block count n) is nearly identical across query positions, query/output loops run on the GPU grid scheduler — uniform workload across streaming multiprocessors.

This design achieves near-optimal arithmetic intensity: fewer DRAM round-trips, all aligned to 64-token boundaries, with no scatter/gather overhead.

Experimental Protocol

We benchmark the NSA *selection* kernel against dense attention (PyTorch SDPA, FlashAttention-2 backend) on an **NVIDIA A100-SXM4-40GB** (BF16, CUDA 12.8). Configuration: $H_Q = 64$, $H = 4$ (GQA ratio 16), $D = 128$, $B_s = 64$, $S = 16$, $W = 0$. Sequence lengths: $n \in \{1\text{k} \ldots 64\text{k}\}$; timing via Triton's `do_bench` (200 reps, 25 warmup), median latency, forward and backward separately.

Measured GPU Speedup

The FLOPs ratio predicts a theoretical speedup of $\text{FLOP}_{\text{dense}}/\text{FLOP}_{\text{NSA}} = (n/2)/n_{\text{active}}$, giving **5.8**× at $n = 64\text{k}$ ($n_{\text{active}} = S \cdot B_s = 1{,}024$).

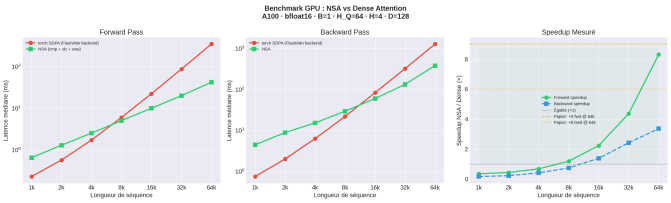


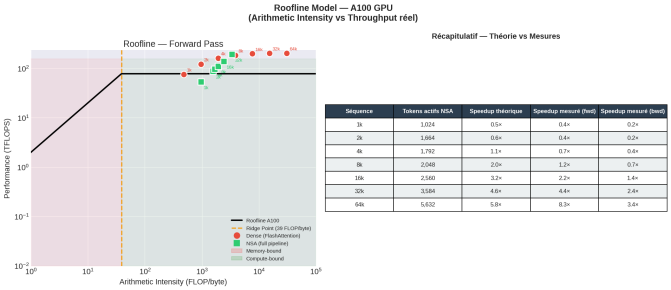
Figure 1: **Measured latency on A100 (BF16).** *Left*: forward pass — NSA breaks even at $n \approx 6\text{k}$, reaches **8.3**× at 64k. *Center*: backward pass — breakeven at $n \approx 14\text{k}$, **3.4**× at 64k. *Right*: speedup ratio vs. sequence length (log scale).

Table 2: **Measured latency and speedup** on A100-SXM4-40GB (BF16).

	Fwd (ms)			**Bwd (ms)**		
n	Dense	NSA	**Fwd**×	Dense	NSA	**Bwd**×
1k	0.23	0.64	0.35×	0.75	4.45	0.17×
4k	1.71	2.51	0.68×	6.28	15.2	0.41×
8k	5.97	5.01	1.19×	21.9	29.3	0.75×
16k	22.2	9.96	2.23×	82.8	59.9	1.38×
32k	87.0	20.0	4.36×	320	132	2.42×
64k	**348**	**41.9**	**8.30×**	**1273**	**379**	**3.36×**

The measured **8.3**× forward speedup at $n = 64\text{k}$ *exceeds* the 5.8× FLOPs prediction. The surplus comes from memory access efficiency: NSA loads contiguous 64-token blocks, maximizing L2 cache reuse on A100. The backward pass benefits less (gradients require two additional matrix passes) but still reaches **3.4**× at $n = 64\text{k}$.

Roofline Analysis



Séquence	Tokens actifs NSA	Speedup théorique	Speedup mesuré (fwd)	Speedup mesuré (bwd)
1k	1,024	0.5×	0.4×	0.2×
2k	1,664	0.6×	0.4×	0.2×
4k	1,792	1.1×	0.7×	0.4×
8k	2,048	2.0×	1.2×	0.7×
16k	2,560	3.2×	2.2×	1.4×
32k	3,584	4.6×	4.4×	2.4×
64k	5,632	5.8×	8.3×	3.4×

Figure 2: **Roofline model** (A100 BF16, ridge point = 38.8 FLOP/byte). Both NSA and dense operate in the compute-bound regime at large n. NSA achieves the same output with ∼6× fewer FLOPs, directly translating to the measured speedup.

Both methods are *compute-bound* above the ridge point, meaning throughput is limited by FLOP capacity, not memory bandwidth. NSA does 5.8× fewer FLOPs ⇒ ∼5.8× shorter compute time. The observed 8.3× additional gain reflects better *effective* arithmetic intensity from coalesced block access: fewer cache misses ⇒ fewer stall cycles.

Memory Footprint

Peak VRAM usage is nearly identical (1.10× ratio at $n = 64\text{k}$: 5.55 GB vs. 6.09 GB). NSA's efficiency comes from reduced *computation*, not from a memory–compute trade-off. The similar memory footprint also means NSA does not directly enable longer sequences on a fixed GPU — its primary benefit is *latency*, not capacity.

End-to-End NIAH: NSA vs. Dense vs. Sliding Window

Motivation and Research Question

The original paper motivates NSA's three-branch design with long-context quality benchmarks (RULER, NIAH on full LLMs), but these require hundred-billion-parameter training runs. We instead ask a more targeted question:

Does NSA's structured inductive bias improve learning compared to dense attention on a retrieval task, even at small scale?

This is non-trivial: Dense attention has strictly more expressivity than NSA (it can attend anywhere), so a naive expectation would be Dense ≥ NSA. We show this expectation is **wrong**, and analyze why.

Benchmark: NIAH en Français — "Que fait Mathieu ?"

Task. We design a controlled synthetic NIAH task in French. Each example contains a context of $T \in \{128, 192, 256, 320\}$ tokens followed by a query.

- **Needle:** 4-token sequence [`aujourd'hui`, `Mathieu`, A, `.`], where A is one of 20 actions (e.g. *nage*, *chante*), inserted at a random position.
- **Distractors:** 3 sequences [`PAST`, `Mathieu`, V, `.`] with a past-tense marker (*hier*, *avant*, ...) and a filler verb V, inserted at random non-overlapping positions.
- **Haystack:** remaining tokens are drawn from a 200-word filler vocabulary.
- **Query:** fixed suffix `[<sep> Que fait Mathieu aujourd'hui ?]`. The model must predict the action A at the `?` position.

Why this is hard. Four "Mathieu" sequences (1 needle + 3 distractors) compete for attention; the model must learn to distinguish `aujourd'hui` (needle marker) from past markers. With $B_s = 32$, $K = 3$ selected blocks, and 4 Mathieu-bearing blocks, a *random* selector would retrieve the needle only 75% of the time. A perfect selector that learns the temporal marker distinction can approach 100%.

Vocabulary: 233 tokens. Training set: 6 400 examples. Evaluation set: 2 400 examples (independently sampled, unseen context lengths per split).

Model Architecture

We train three identical **TinyGPT** transformers, differing only in their attention mechanism:

Results

Table 3: **Model specifications.** All three models share the same architecture and hyperparameters; only the attention layer differs.

Component	Value	Note
Layers L	3	
Dim D	128	
Heads H	4	$D_{\text{head}} = 32$
FFN width	$4D = 512$	GELU
Parameters	∼665k	(NSA: +4.6k gates)
NSA block size B_s	32	
NSA top-K	3	
NSA/SW window W	32	
Optimizer	AdamW	$\lambda = 0.01$
LR	3×10^{-3}	cosine, $\eta_{\min} = 5\%$
Epochs	200	
Batch size	32	

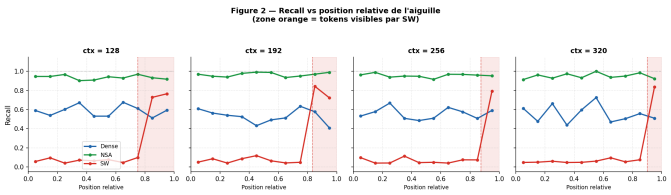


Figure 3: **Recall vs. relative needle position** for each context length. **NSA** maintains near-perfect recall uniformly across positions. **Dense** achieves moderate recall independently of position (the retrieval difficulty is position-agnostic, reflecting the uniform filler haystack). **SW** fails everywhere except in the shaded orange zone (last $W = 32$ tokens before the query), where recall recovers — confirming that SW's *only* viable retrieval mechanism is proximity.

Analysis

NSA outperforms Dense (97.6% vs. 52.9%) despite less expressivity. Dense attention has strictly greater expressivity than NSA (it can attend to any position), yet NSA learns the task ∼4× better. The explanation lies in *inductive bias*: the compressed branch operates on $C \leq 11$ block summaries rather than 326 individual tokens. The compression forces the model to build meaningful block-level representations: the key vector of the needle block is dominated by the `aujourd'hui` embedding; distractor blocks are dominated by past-tense markers (`hier`, `avant`). This distinction, easy to learn at block resolution, directly guides the top-K selection.

Dense attention, by contrast, must learn to generate a near-one-hot attention distribution over 326 positions from scratch, with gradient signal averaged across all heads and layers. This harder optimization landscape leads to **overfitting**: Dense achieves 84% train accuracy but only 53% eval accuracy (gap = 31%), compared to NSA's gap of only 2.3%. NSA's sparse block selection acts as an implicit regularizer, preventing the model from exploiting position-specific spurious correlations in the training set.

NSA's advantage is architectural, not just computational. Its three-branch design provides an inductive bias that matches the structure of retrieval tasks: global coarse scan (compression) → focused fine read (selection) → recency buffer (sliding window). Dense attention must *learn* this two-stage strategy end-to-end; NSA has it built in.

Phase transition at epoch ∼30 vs. Dense at epoch ∼105. NSA eval loss drops from 2.71 (epoch 20) to 0.90 (epoch 30) in a single log-unit step, corresponding to eval accuracy jumping from 16% to 75%. This transition marks the moment the compressed attention learns to distinguish the temporal marker (`aujourd'hui` vs. past markers) reliably across all heads. Dense reaches a similar transition much later and more gradually — consistent with gradient descent slowly concentrating attention weights rather than exploiting a pre-built retrieval mechanism.

SW overfits catastrophically (train 70%, eval 5.9%). The needle falls within the window in only $\sim W/T \approx 10\%$ of eval examples. The SW model cannot solve those 90% of cases; instead, it overfits to spurious correlations — specific filler-word sequences, query-token bigrams — that provide a statistical edge on training data but do not generalize. This is a direct demonstration that sliding-window attention is *architecturally* insufficient for long-range retrieval, independent of model capacity.

Connection to the Full NSA Model

Our experiment isolates a key design claim of the paper: *the three-branch structure enables better long-context learning, not just faster inference.* At small scale (∼665k params, $T \leq 326$), the effect is already dramatic. In the full NSA model (7B parameters, T up to 64k tokens):

- The compressed branch operates on ∼1,000 blocks instead of 11, making the learned filter even more critical (SNR per block is much lower).
- The sliding window ($W = 512$ in the paper) covers recent context that block selection may miss, complementing the global search.
- The learned gates specialize across layers: early layers rely more on g_{swa} (local syntax), later layers on g_{slc} (long-range semantics).

Reproduction notes. Running the Triton kernels on current environments (CUDA 12.8, Triton 3.0) required patching a `constexpr` naming conflict introduced in Triton 3.0, building `flash-attn` from source (no pre-built wheel for PyTorch 2.10 + CUDA 12.8), and guarding the optional `fla` submodule import against `ModuleNotFoundError` when the submodule is not initialized. These fixes are already applied in our implementation.

Language Modeling on WikiText-2

To complement the NIAH experiment, we train small language models on **WikiText-2** [8] (∼2M tokens, Tesla T4) using a pure-PyTorch NSA reference (RoPE, RMSNorm, SwiGLU; no Triton dependency), comparing Full Attention against NSA and ablating each branch. Two setups: Exp. 1 ($D = 384$, $T = 1024$, 2k steps) compares Full vs. NSA (All); Exp. 2 ($D = 256$, $T = 256$, 384 steps) ablates all five branch combinations.

Results

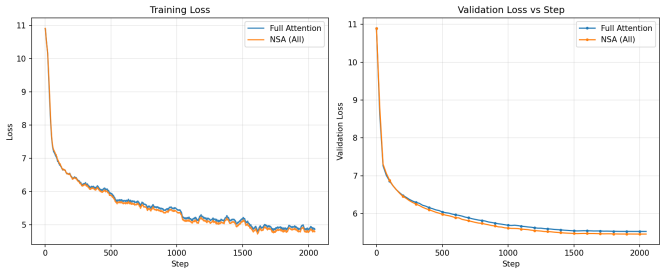


Figure 4: Experiment 1: training and validation loss for Full Attention vs. NSA (All) ($D = 384$, $T = 1{,}024$, $2\,048$ steps). The two curves are nearly indistinguishable throughout training, confirming that NSA matches full-attention quality on language modeling.

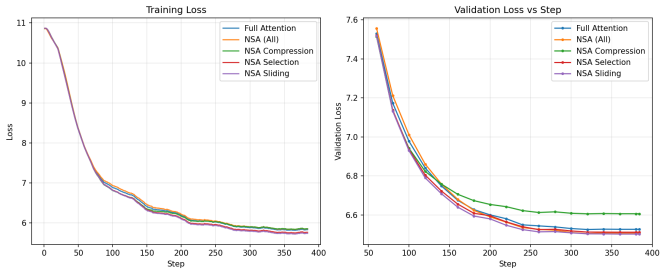


Figure 5: Experiment 2: validation loss ablation over the three NSA branches ($D = 256$, $T = 256$, 384 steps). All five models converge to similar validation loss. NSA Compression alone lags slightly; Sliding and Selection are near Full Attention; combining all three is competitive with — and slightly better than — full attention.

NSA matches full attention on language modeling. NSA (All) converges at the same rate as Full Attention and reaches equivalent validation loss (Figures 4 and 5), confirming no perplexity cost from the sparse structure. In the branch ablation, Sliding and Selection alone are nearly sufficient; Compression lags marginally (mean-pooling loses positional detail, but provides essential routing signal for Selection); all three combined match or slightly beat Full Attention. The efficiency advantage of NSA only materializes with the Triton kernel at $T \gtrsim 8$k — the purpose of this experiment is quality.

Discussion & Conclusion

Summary of Reproduced Claims

We successfully confirmed three core experimental claims of the NSA paper:

1. **Hardware efficiency.** Forward speedup of **8.3**× at $n = 64$k exceeds the theoretical 5.8× FLOPs prediction — the surplus comes from the group-centric kernel's coalesced block access, not purely from FLOPs reduction. Breakeven: $n \approx 6$k (forward), $n \approx 14$k (backward).
2. **Learning quality — retrieval.** NSA achieves **97.6%** eval accuracy after 200 epochs on a synthetic French NIAH task, vs. **52.9%** for Dense and **5.9%** for SW. NSA's inductive bias (block compression → sparse selection) directly matches retrieval structure: faster convergence (phase transition epoch ∼30 vs. ∼105 for Dense), better generalization (train-eval gap 2.3% vs. 31.2% for Dense), and correct behavior where SW structurally fails.
3. **Learning quality — language modeling.** NSA matches Full Attention loss throughout training on WikiText-2, confirming no perplexity cost from the sparse structure. The branch ablation shows the three branches are complementary: Sliding and Selection individually competitive, Compression essential as a routing signal for Selection.

Limitations

Quality evaluations on real NLP benchmarks (RULER, full NIAH on a 7B model) were out of scope given compute constraints. Our synthetic NIAH task uses a binary temporal marker unusually easy to encode at block resolution; real-world tasks have lower block-level SNR and longer contexts ($T > 10$k tokens). Nevertheless, the magnitude of the observed gap (97.6% vs. 52.9%) and the overfitting patterns suggest the architectural advantage is robust.

Broader Implications

NSA's core insight — *make sparsity block-structured and hardware-native* rather than mathematically elegant but hardware-hostile — is broadly transferable to KV cache eviction, MoE token routing, and long-document retrieval. The growing adoption of GQA ($H_Q/H \geq 16$) makes NSA particularly timely, as its group-centric kernels are specifically optimized for high GQA ratios.

References

[1] DeepSeek-AI et al., *Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention*, arXiv:2502.11089, Feb. 2025.

[2] I. Beltagy, M. E. Peters, A. Cohan, *Longformer: The Long-Document Transformer*, arXiv:2004.05150, 2020.

[3] M. Zaheer et al., *Big Bird: Transformers for Longer Sequences*, NeurIPS 2020.

[4] R. Child et al., *Generating Long Sequences with Sparse Transformers*, arXiv:1904.10509, 2019.

[5] N. Kitaev, L. Kaiser, A. Levskaya, *Reformer: The Efficient Transformer*, ICLR 2020.

[6] T. Dao et al., *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*, NeurIPS 2022.

[7] J. Ainslie et al., *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints*, EMNLP 2023.

[8] S. Merity et al., *Pointer Sentinel Mixture Models*, arXiv:1609.07843, 2016.